

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 5 September 2017 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

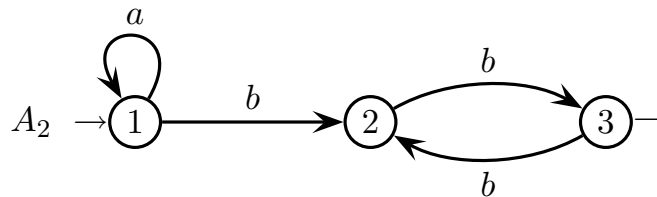
1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$ and answer the following questions:

(a) Consider the following regular expression R_1 , over the alphabet Σ :

$$R_1 = (a a)^* b^+$$

By using the Berry-Sethi (BS) method, find a deterministic automaton A_1 equivalent to the regular expression R_1 . If necessary minimize the automaton A_1 , or reasonably justify that it is already minimal.

(b) Consider the following automaton A_2 , over the alphabet Σ :



Find a (possibly non-deterministic) automaton A_3 that recognizes the union language $L(A_1) \cup L(A_2)$. This automaton A_3 may have ε -transitions (spontaneous transitions), and must have only one initial state and only one final state. Then eliminate all the ε -transitions and obtain a (possibly non-deterministic) automaton, still with only one initial state and only one final state.

- (c) By using the node elimination (Brzozowsky-McCluskey - *BMC*) method, find a regular expression R_3 equivalent to the automaton A_3 .
- (d) (optional) Determine if the regular expression R_3 obtained at point (c) is ambiguous or not, and reasonably justify your answer. In case of ambiguity, find an equivalent yet unambiguous regular expression R_4 and reasonably justify it.
-

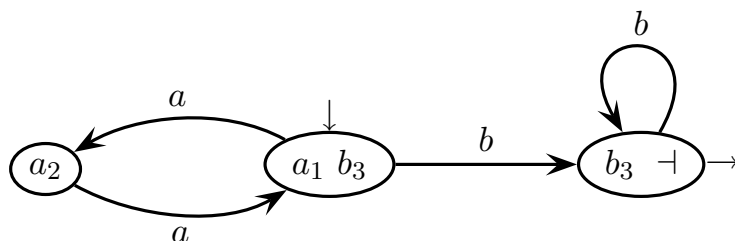
Solution

- (a) Here is the construction of the *BS* automaton A_1 :

$$R_{1,\#} = (a_1 a_2)^* b_3^+ \neg$$

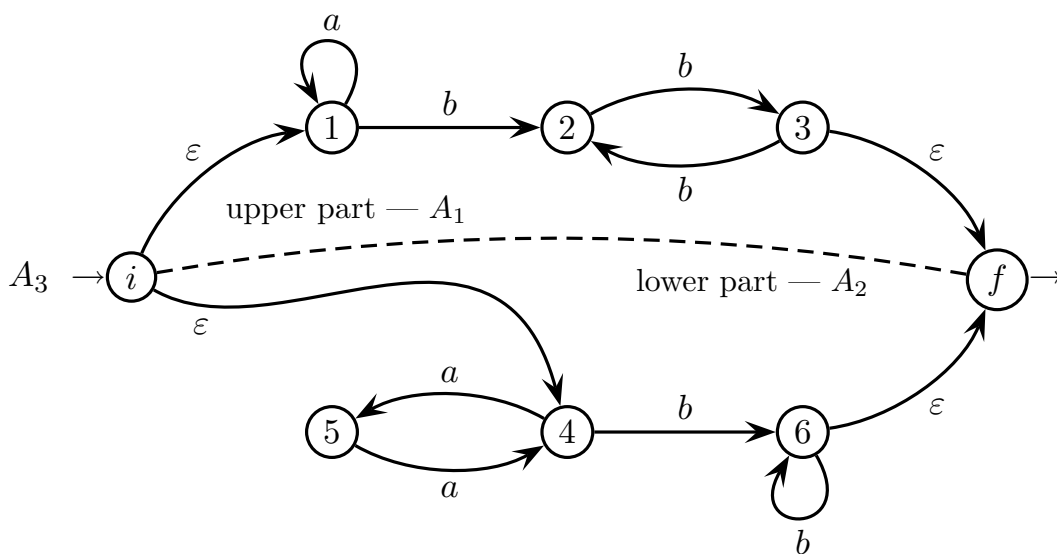
initials	a_1, b_3
terminals	followers
a_1	a_2
a_2	a_1, b_3
b_3	$b_3, \neg$

Then, here is the state-transition graph of the deterministic automaton A_1 produced by the *BS* method, with three states:

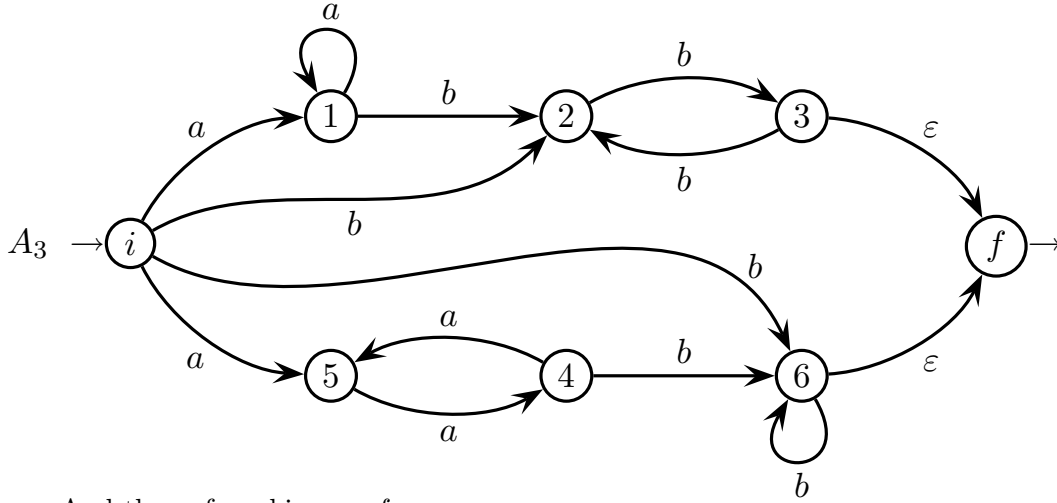


By construction, automaton A_1 is deterministic and in clean form. It is already minimal, as the two non-final states have differently labeled outgoing arcs and so are distinguishable.

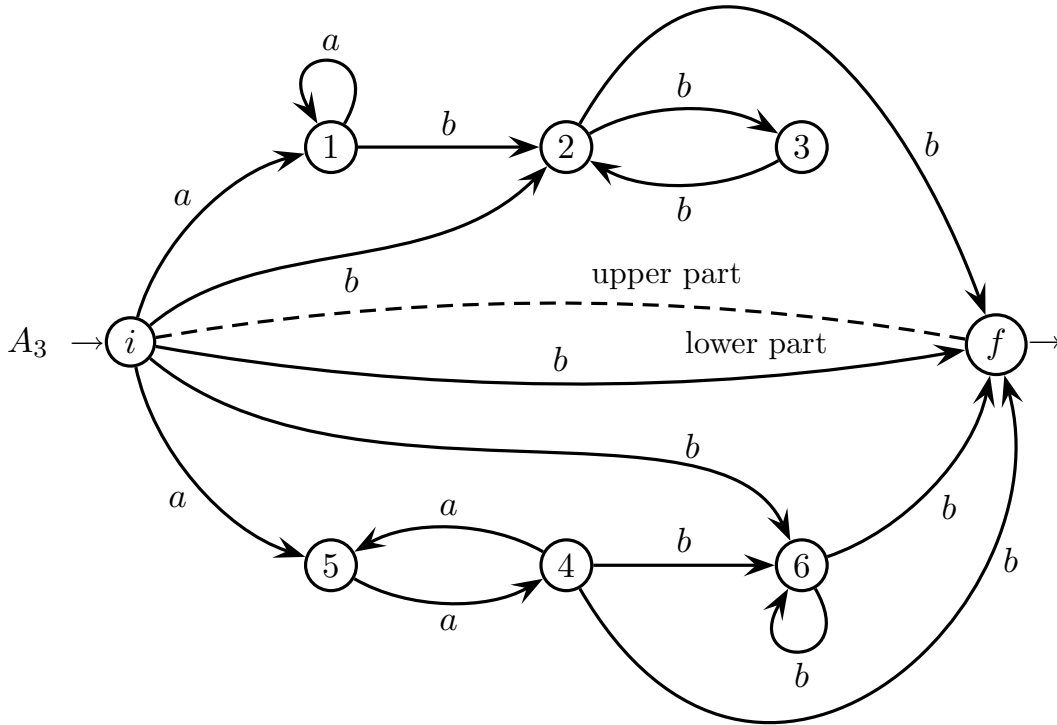
- (b) Here is the union automaton A_3 , obtained by the Thompson method, with eight states in total (state i is initial and state f is final):



Eliminating the ε -transitions is simple. By using back-propagation to cut those leaving the initial state i and forward-propagation to cut those entering the final state f , at one time we obtain that the initial and final states i and f remain unique. Here is the result of first working on i :



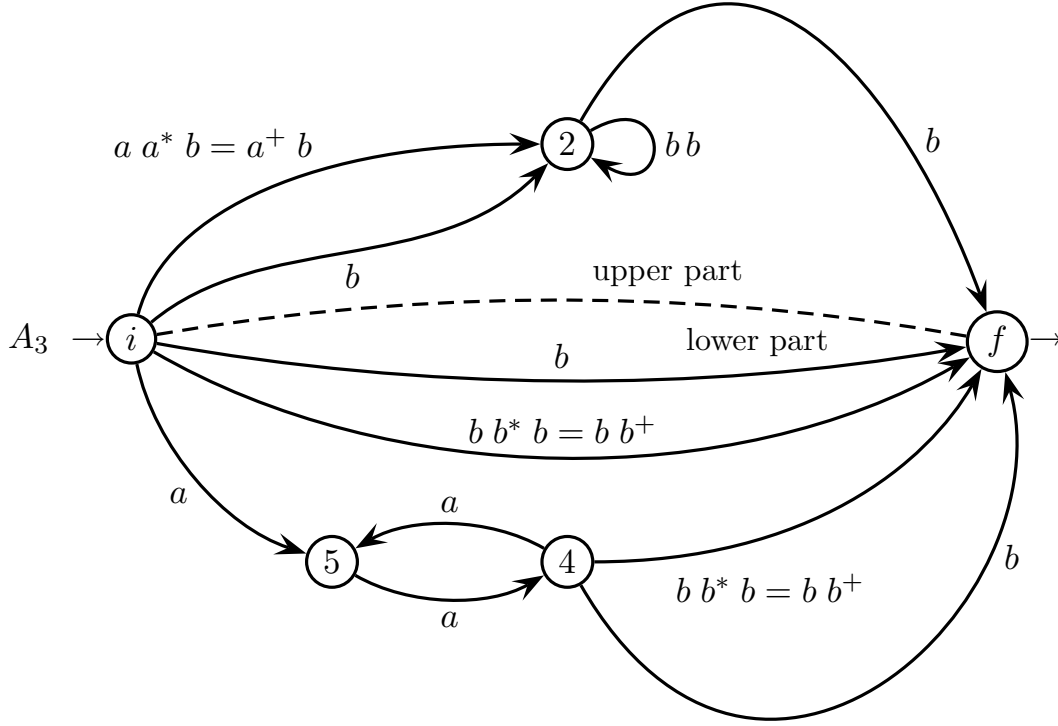
And then of working on f :



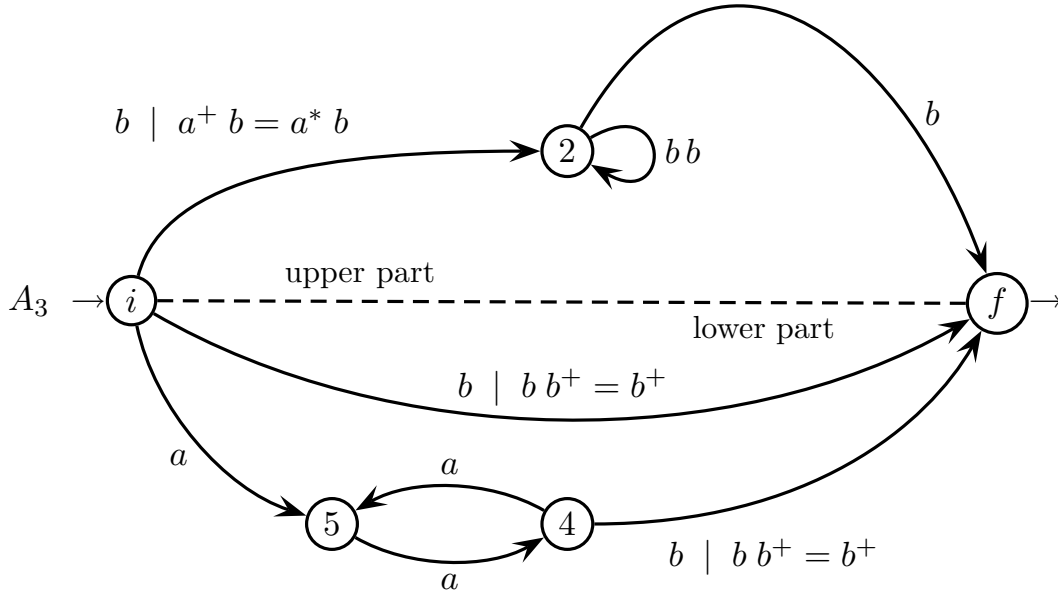
This is the requested automaton A_3 : it does not have any spontaneous transitions, and it has unique initial and final states; though, it is still non-deterministic.

- (c) Node elimination can be easily applied to the automaton A_3 (with or without ε -transitions), since automaton A_3 already has one initial and one final state, the initial one does not have any ingoing transitions and the final one does not have any outgoing transitions.

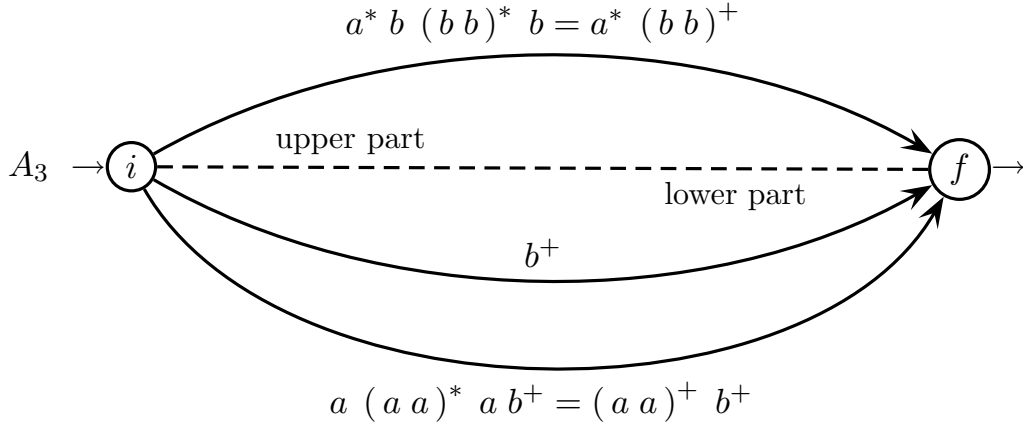
We compact the elimination steps, as the two parts do not have any cross-connections. First remove at one time but separately nodes 1 and 3 in the upper part, as they are loosely connected, and remove node 6 in the lower part:



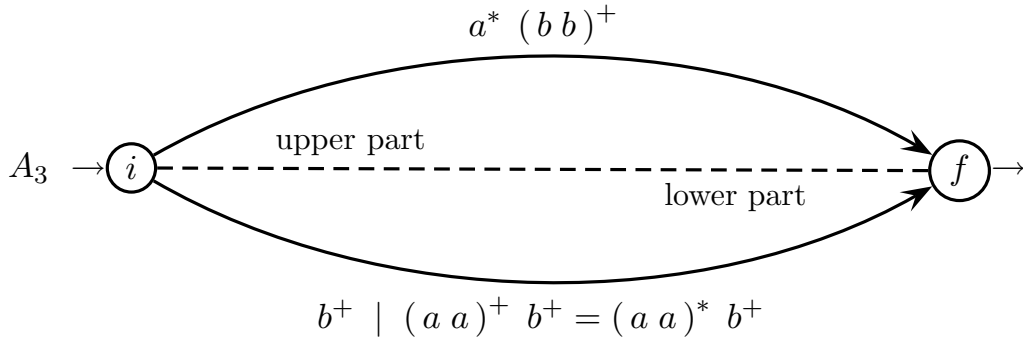
and unify the bundles of arcs:



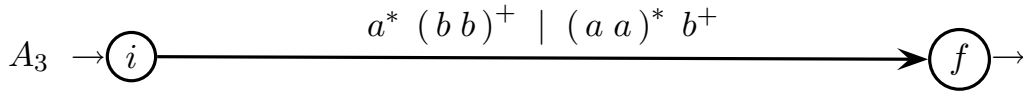
Then remove node 2 in the upper part, and remove together nodes 4 and 5 in the lower part, as they are tightly connected:



and unify the bundle of two arcs in the lower part:



Finally unify the two parts:



and obtain an equivalent regular expression:

$$R_3 = \underbrace{a^* (b b)^+}_{L(A_1)} \mid \underbrace{(a a)^* b^+}_{L(A_2)}$$

Notice that the left member is (equivalent to) language $L(A_1)$ and that the right member is (equivalent to) language $L(A_2)$.

- (d) The regular expression R_3 is ambiguous, since the strings bb and $aabb$ are generated by both members of the union. In fact, number the terminals in R_3 :

$$R_{3,\#} = a_1^* (b_2 b_3)^+ \mid (a_4 a_5)^* b_6^+$$

and notice that the numbered $R_{3,\#}$ generates $b_2 b_3$ or $b_6 b_6$, and $a_1 a_1 b_2 b_3$ or $a_4 a_5 b_6 b_6$. So the regular expression R_3 exhibits union ambiguity and in general all the strings of type $a^{2h} b^{2k}$ with $h, k \geq 0, 1$ are ambiguous. This comes from that the intersection of languages $L(A_1)$ and $L(A_2)$ is not empty.

Since the intersection language of $L(A_1)$ and $L(A_2)$ is $a^{2h} b^{2k}$ with $h, k \geq 0, 1$, or equivalently it is $(aa)^* (bb)^+$, an obvious way to disambiguate the regular expression R_3 consists of preventing its right member $(aa)^* b^+$ from generating the strings with an even number $(0, 2, 4, \dots)$ of letters b (without changing the letters a). Here is a possible formulation:

$$R_4 = a^* (bb)^+ \mid (aa)^* b (bb)^*$$

The regular expression R_4 is equivalent to R_3 , yet it is not ambiguous as it cannot generate strings bb or $aabb$ (or all the other previously ambiguous strings) in more than one way.

Alternatively, we could prevent the left member $a^* (bb)^+$ of R_3 from generating the strings with an even number $(0, 2, 4, \dots)$ of letters a (without changing the letters b). So here is a possible variant:

$$R_4 = a (aa)^* (bb)^+ \mid (aa)^* b^+$$

This variant for R_4 is also equivalent to R_3 , yet it is not ambiguous as it cannot generate strings bb or $aabb$ (or all the other previously ambiguous strings) in more than one way.

2 Free Grammars and Pushdown Automata 20%

1. Consider a two-letter alphabet $\Sigma = \{ a, b \}$ and answer the following questions:

(a) Define a *BNF* grammar G_1 that generates the following language L_1 :

$$L_1 = \{ a^i b^j \mid i, j \geq 0 \text{ and } i \neq j \}$$

(b) Draw a syntax tree of the grammar G_1 for the valid string $a a a b \in L_1$.

(c) Say if grammar G_1 is ambiguous or not, and adequately justify your answer. In case it is ambiguous, write an equivalent yet unambiguous *BNF* grammar G'_1 .

(d) (optional) Define a *BNF* grammar G_2 that generates the following language L_2 :

$$L_2 = \{ a^i b^j c^k \mid i, j, k \geq 0 \text{ and } (i \neq j \text{ or } j \neq k) \}$$

Notice that the alphabet of language L_2 has three letters: a , b and c .

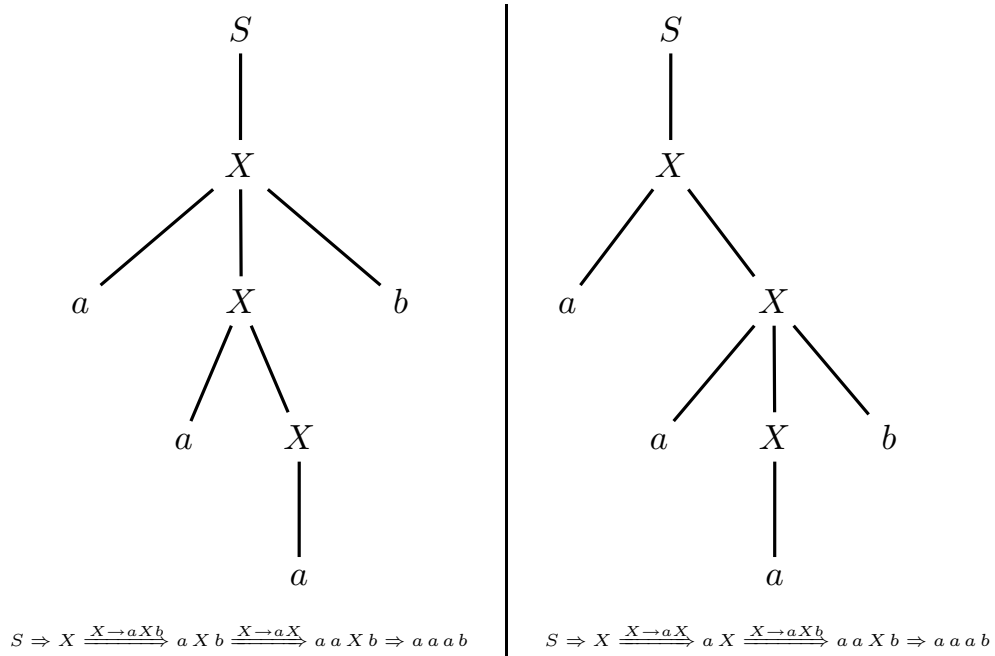
Solution

(a) Here is a possible *BNF* grammar G_1 (axiom S):

$$G_1 \left\{ \begin{array}{l} \hline S \rightarrow X \mid Y \quad \text{--- top rule, two cases} \\ \hline X \rightarrow a \mid aX \mid aXb \quad \text{--- case } i > k \geq 0 \\ \hline Y \rightarrow b \mid Yb \mid aYb \quad \text{--- case } 0 \leq i < k \\ \hline \end{array} \right.$$

which is somewhat intuitive; see point (b) for a correctness test.

(b) Here are two trees for the valid string $aaab \in L_1$:



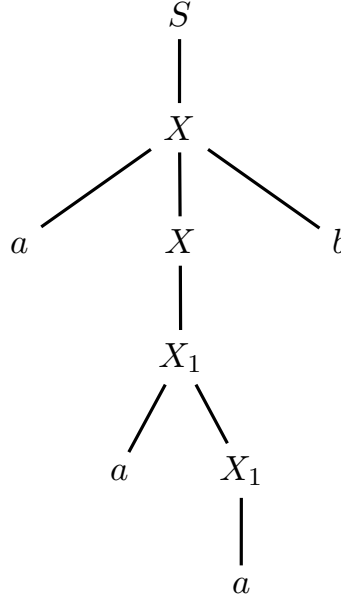
The string has two trees as in its derivation the rules $X \rightarrow aX$ and $X \rightarrow aXb$ can be commuted without changing the final result. A similar situation occurs with the valid string $abbb \in L_1$. Longer strings have more and more trees.

(c) Grammar G_1 is ambiguous, see point (b). An equivalent yet unambiguous grammar G'_1 is the following (axiom S):

$$G'_1 \left\{ \begin{array}{l} \hline S \rightarrow X \mid Y \quad \text{--- top rule, two cases} \\ \hline X \rightarrow aXb \mid X_1 \quad \text{--- case } i > k \geq 0 \\ X_1 \rightarrow a \mid aX_1 \\ \hline Y \rightarrow aYb \mid Y_1 \quad \text{--- case } 0 \leq i < k \\ Y_1 \rightarrow b \mid Y_1b \\ \hline \end{array} \right.$$

The idea is that the applications of rules $X \rightarrow a X$ and $X \rightarrow a X b$ are ordered in a fixed way in the derivation of G'_1 : first all the applications of $X \rightarrow a X b$, second all those of $X \rightarrow a X$, by renaming X to X_1 (fixing the opposite order would work as well). Similarly for the nonterminal Y .

Now the valid string $a a a b \in L_1$ has only one tree:



which corresponds to the left one above, whereas the right one is impossible.

- (d) Here is a possible *BNF* grammar G_2 (axiom S), a little more complicated than grammar G_1 , but based on the same principle:

$$G_2 \left\{ \begin{array}{ll} S \rightarrow A X \mid Y C & \text{--- top rule, two cases} \\ \hline A \rightarrow a A \mid \varepsilon & \text{--- case } j \neq k \\ X \rightarrow X_1 \mid X_2 & \\ X_1 \rightarrow b \mid b X_1 \mid b X_1 c & \text{--- subcase } j > k \\ X_2 \rightarrow c \mid X_2 c \mid b X_2 c & \text{--- subcase } j < k \\ \hline C \rightarrow c C \mid \varepsilon & \text{--- case } i \neq j \\ Y \rightarrow Y_1 \mid Y_2 & \\ Y_1 \rightarrow a \mid a Y_1 \mid a Y_1 b & \text{--- subcase } i > j \\ Y_2 \rightarrow b \mid Y_2 b \mid a Y_2 b & \text{--- subcase } i < j \end{array} \right.$$

Grammar G_2 is ambiguous, for the same reasons as those of grammar G_1 and for one more: that a string satisfying both $j \neq k$ and $i \neq j$ can be generated both by starting with rule $S \rightarrow A X$ and by starting with rule $S \rightarrow Y C$.

2. Consider a programming language the phrases of which are set-theoretic expressions, defining sets of numbers. Below are the composition rules of such expressions:

- An element of a set is an integer in base 10, greater than or equal to zero, unsigned and without leading digits 0.
- A variable of type set is an alphabetical identifier of this type: it may be any sequence (of length ≥ 1) of uppercase letters and underscore signs ‘_’, it must begin and terminate by a letter, and it may not contain adjacent underscores.
- A set is a list, enclosed by graph parentheses ‘{’ and ‘}’, of objects of these four types:
 - elements
 - variables
 - sets
 - expressions

in any arrangement, separated by commas ‘,’; the empty set is also allowed.

- An expression may contain the following operators, all applicable to variables, sets and subexpressions: union denoted as ‘ $\cup$ ’ and intersection denoted as ‘ $\cap$ ’, both written in the infix position; and complement denoted as ‘ $\neg$ ’, written in the prefix position.
- Both the union and the intersection operators are left-associative.
- The syntax does not specify any precedence order between the union and intersection operators, and they are simply computed from left to right, but the complement operator takes precedence over both union and intersection.
- A (sub)expression may contain subexpressions, delimited by the round parentheses ‘(’ and ‘)’, at any depth level.
- A language phrase is a single set-theoretic expression, as defined above.

Further detailed information about the lexical and syntactic structure of the programming language can be inferred from the list of six sample expressions below (each of them is a single language phrase):

$$\{ \} \tag{1}$$

$$\{ 1, 20 \} \tag{2}$$

$$\{ A, 23, A \cup B \cup SET \} \tag{3}$$

$$\{ B \cap A, 32, \{ 3, \neg C \}, \{ \}, ZZZ \} \tag{4}$$

$$A \cup SET_A \cap \neg C \cup \neg \{ A \cup B, 45 \} \tag{5}$$

$$\{ 6708, \neg (A \cup B) \} \tag{6}$$

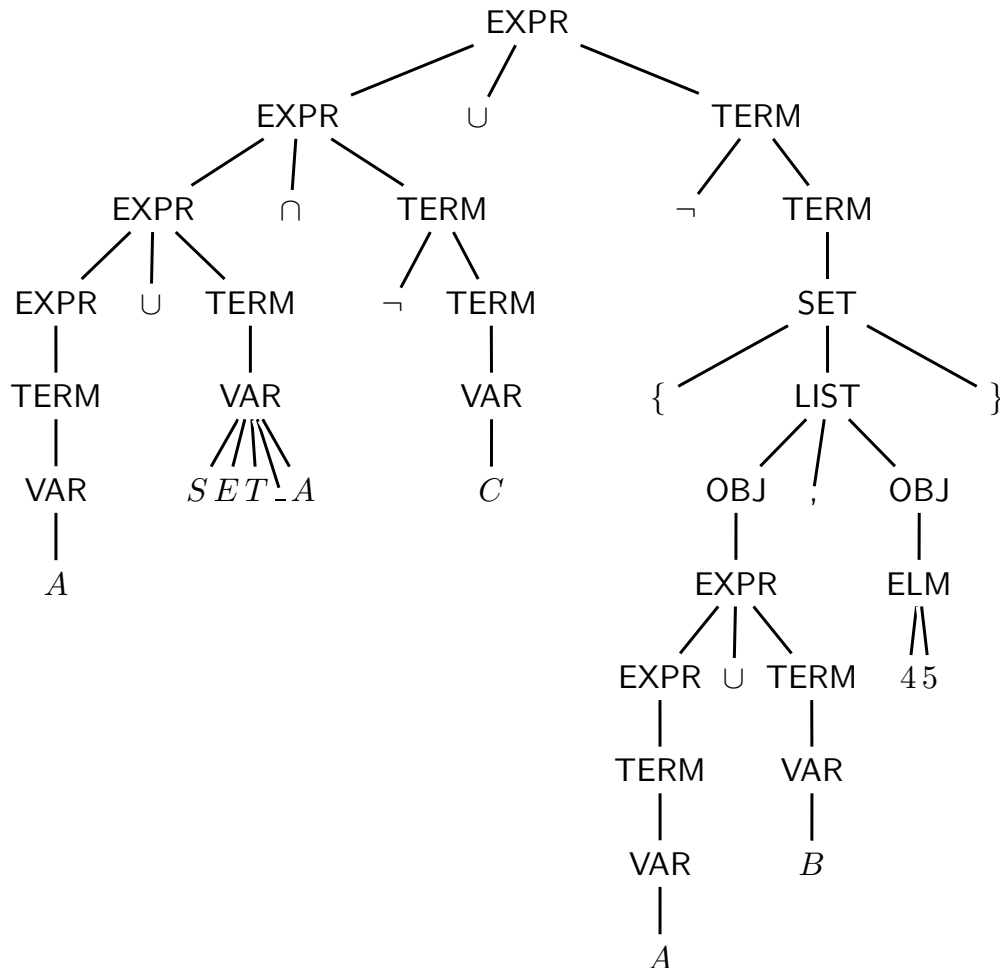
Write a grammar, not ambiguous, that models this kind of set-theoretic expression. Test your grammar and draw the syntax tree of the sample phrase (5).

Solution

Here is a possible grammar (axiom $\langle \text{EXPR} \rangle$), with a mix of *BNF* and *EBNF* rules:

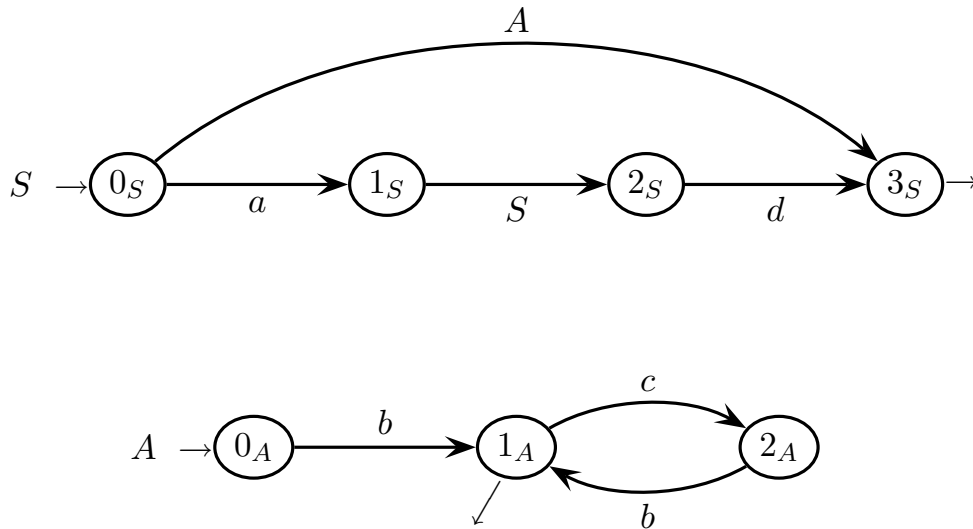
$$\left\{ \begin{array}{l} \langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle (\cup \mid \cap) \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle \rightarrow \neg \langle \text{TERM} \rangle \mid (\langle \text{EXPR} \rangle) \mid \langle \text{SET} \rangle \mid \langle \text{VAR} \rangle \\ \hline \langle \text{SET} \rangle \rightarrow \{ [\langle \text{LIST} \rangle] \} \\ \langle \text{LIST} \rangle \rightarrow \langle \text{OBJ} \rangle (, \langle \text{OBJ} \rangle)^* \\ \langle \text{OBJ} \rangle \rightarrow \langle \text{EXPR} \rangle \mid \langle \text{ELM} \rangle \\ \hline \langle \text{VAR} \rangle \rightarrow [A - Z]^+ (_ [A - Z]^+)^* \\ \langle \text{ELM} \rangle \rightarrow [1 - 9] [0 - 9]^* \mid 0 \end{array} \right.$$

The grammar is not ambiguous. It is simple, but wants understanding. Union and intersection are modeled by a left-recursive *BNF* rule, since they are left-associative (here using *EBNF* rules would be wrong as such rules do not specify an association order) and they do not have any specified precedence level, while complement is at a level higher than both. The rest of the grammar can be modeled by *EBNF* rules (of course the solution is not unique). Here is the test tree for the sample expression (5):



3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the two-letter nonterminal alphabet $V = \{ S, A \}$ (axiom S):



Answer the following questions (for each of them, space is prepared on the next pages):

- (a) Draw the complete pilot of grammar G , show that grammar G is of type $ELR(1)$ and shortly justify your answer.
- (b) Simulate the ELR analysis of the following valid sample string:

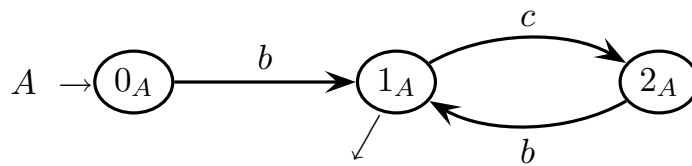
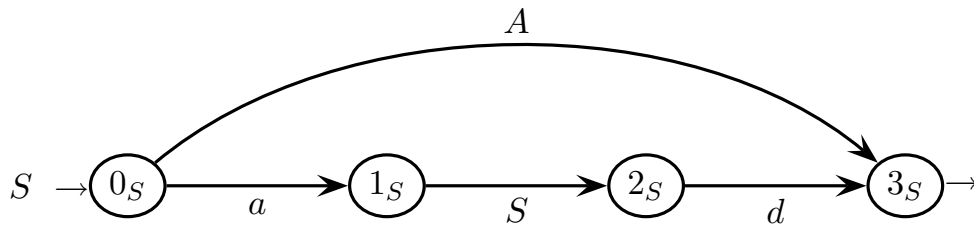
$$a b c b d \in L(G)$$

Please use the table prepared on the next page.

- (c) Find all the guide sets on the arcs of the machine net of grammar G (use the net on the next page) and argue that grammar G is of type $ELL(1)$.
- (d) (optional) Write the syntactic procedures of nonterminals S and A for the recursive descent syntax analyzer of grammar G .

here draw the pilot of grammar G
and show that grammar G is of type $ELR(1)$ - question (a)

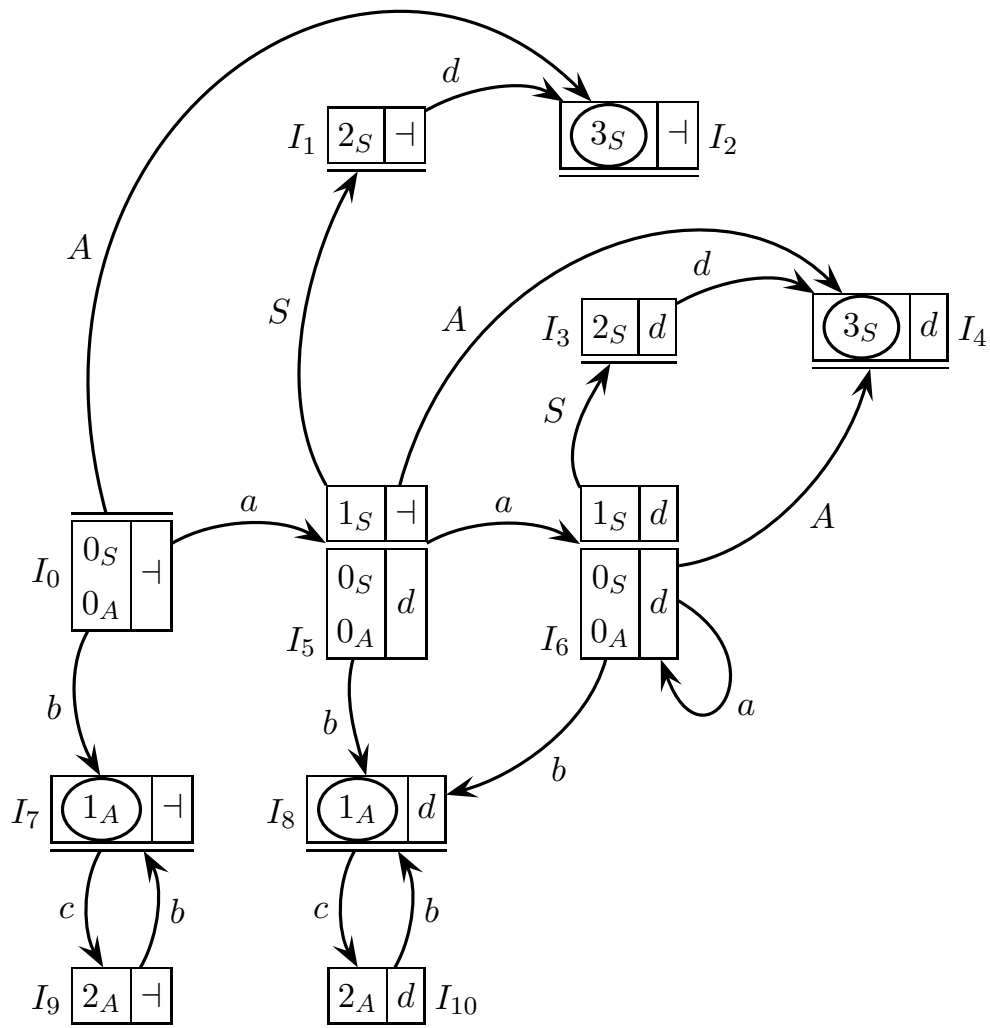
here complete the net of grammar G with the call arcs,
 write the guide sets on all the arcs
 and argue that the grammar is of type $ELL(1)$ - question (c)



here write the syntactic procedures of nonterminals S and A - question (d)

Solution

(a) Here is the complete pilot of grammar G , with 11 m-states (initial m-state I_0):



The pilot does not have any conflicts (shift-reduce, reduce-reduce or convergence). Thus grammar G is of type $ELR(1)$.

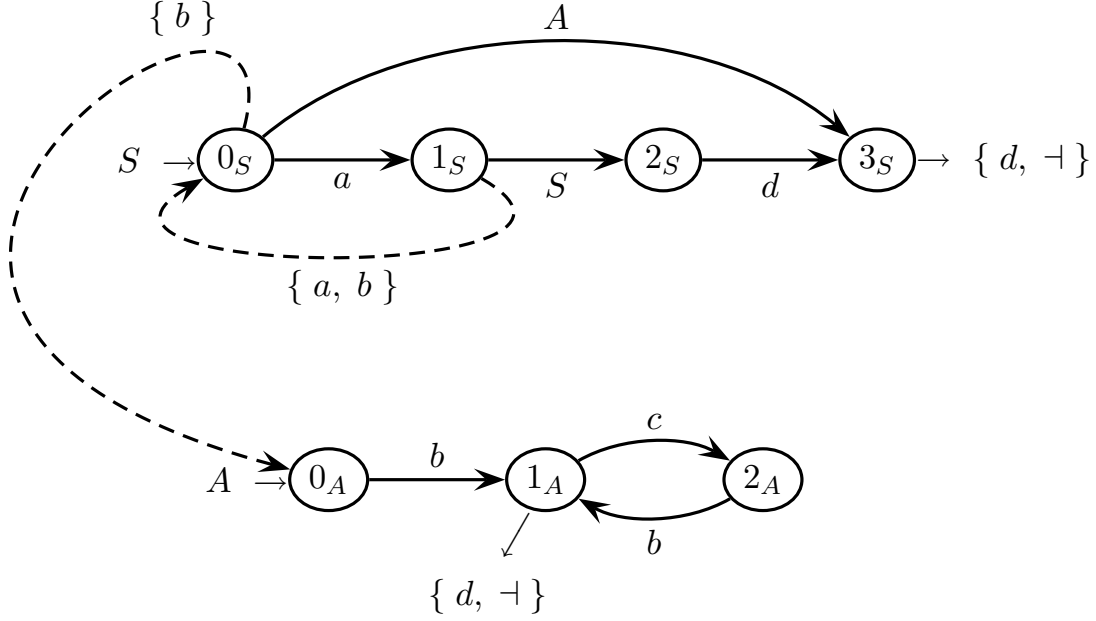
Actually the language is quite simple: $L = \{ a^h b (c b)^k d^h \mid h \geq 0 \wedge k \geq 0 \}$. Though, clearly it is not regular.

(b) Here is the simulation table for the *ELR* analysis of the valid string $a b c b d$:

<i>stack of alternated macro-states and grammar symbols</i>	<i>input left to be scanned</i>	<i>move executed</i>
I_0	$a b c b d \dashv$	initial configuration
$I_0 a I_5$	$b c b d \dashv$	terminal shift: $I_0 \xrightarrow{a} I_5$
$I_0 a I_5 b I_8$	$c b d \dashv$	terminal shift: $I_5 \xrightarrow{b} I_8$
$I_0 a I_5 b I_8 c I_{10}$	$b d \dashv$	terminal shift: $I_8 \xrightarrow{c} I_{10}$
$I_0 a I_5 b I_8 c I_{10} b I_8$	$d \dashv$	terminal shift: $I_{10} \xrightarrow{b} I_8$
$I_0 a I_5$	$d \dashv$	reduction: $b c b \rightsquigarrow A$
$I_0 a I_5 A I_4$	$d \dashv$	nonterminal shift: $I_5 \xrightarrow{A} I_4$
$I_0 a I_5$	$d \dashv$	reduction: $A \rightsquigarrow S$
$I_0 a I_5 S I_1$	$d \dashv$	nonterminal shift: $I_5 \xrightarrow{S} I_1$
$I_0 a I_5 S I_1 d I_2$	$\dashv$	terminal shift: $I_1 \xrightarrow{d} I_2$
I_0	$\dashv$	reduction: $a S d \rightsquigarrow S$
initial stack contents	tape is empty	last move is a reduction to S
stop - acceptance condition is verified - accept		

In the stack we have omitted the pointer chains that are used to identify and pop the handle of a reduction move, as this is an implementation aspect. Furthermore, grammar G is of type *ELL* (1) (see question (c)), thus the stack contains only one analysis thread, which simplifies the identification of a reduction handle.

- (c) Here is the *PCFG* (Parser Control Flow Graph) of grammar G , with all the guide sets (those on the terminal shift arcs coincide with the labeling terminals and here are not shown):



The guide sets on the two bifurcations states 0_S and 1_A are disjoint. Therefore grammar G is of type $ELL(1)$. This is coherent with the fact that the pilot (see question (a)) satisfies the *ELL* condition, as grammar G is not left-recursive, and its pilot is of type *ELR*(1) and has the *STP* since all the bases of the m-states of the pilot contain at most one item. Notice that actually the machine M_A does not call any other machine, while the axiomatic machine M_S calls machine M_A and moreover is recursive (and auto-inclusive).

- (d) The pseudo-code that encodes the syntactic procedures of the nonterminals S and A , obtained by examining each state and faithfully translating each arc, exit arrow and error case, along with the main program, is the following:

```

procedure  $S$ 
if  $cc \in \{ a \}$  then                                // 1-st test in state 0
     $cc = next$                                            // goto state 1
    if  $cc \in \{ a, b \}$  then                            // test in state 1
        call  $S$                                            // goto state 2
        if  $cc \in \{ d \}$  then                            // test in state 2
             $cc = next$                                      // goto state 2
            if  $cc \in \{ d, \vdash \}$  then                // test in state 3
                return                                     // exit from state 3
            else error                                    // error in state 3
        else error                                       // error in state 2
    else error                                           // error in state 1
else if  $cc \in \{ b \}$  then                            // 2-nd test in state 0
    call  $A$                                                // goto state 3
    if  $cc \in \{ d, \vdash \}$  then                        // test in state 3
        return                                           // exit from state 3
    else error                                           // error in state 3
else error                                           // error in state 0

procedure  $A$ 
if  $cc \in \{ b \}$  then                                // test in state 0
     $cc = next$                                            // goto state 1
    while  $cc \in \{ c \}$  do                            // 1-st (cyclic) test in state 1
         $cc = next$                                      // goto state 2
        if  $cc \in \{ b \}$  then                            // test in state 2
             $cc = next$                                      // go back to state 1
        else error                                       // error in state 2
    if  $cc \in \{ d, \vdash \}$  then                        // 2-nd test in state 1
        return                                           // exit from state 1
    else error                                           // error in state 1
else error                                           // error in state 0

program ELL_PARSER
 $cc = next$                                            // read first char or terminator
call  $S$                                                // invoke the axiomatic procedure
if  $cc \in \{ \vdash \}$  then accept                    // acceptance condition
else reject

```

Algorithm: Recursive descent analyzer, encoded from the machine net.

The pseudo-code could be optimized in various ways, saving a few statements.

4 Language Translation and Semantic Analysis 20%

1. Consider a three-letter terminal alphabet $\Sigma = \{0, 1, c\}$ and the following source grammar G , of type *BNF* (axiom X):

$$G \left\{ \begin{array}{l} X \rightarrow 0 X 0 \\ X \rightarrow 1 X 1 \\ X \rightarrow c \end{array} \right.$$

The source grammar G generates the palindromes of bits with centre c .

Answer the following questions:

- (a) Consider the syntactic translation τ_1 of the source language $L(G)$ that outputs the palindromes and strips off the longest suffix of bits 0 (if any). Examples:

$$\begin{aligned} \tau_1(c) &= c & \tau_1(1c1) &= 1c1 & \tau_1(00c00) &= 00c \\ \tau_1(01c10) &= 01c1 & \tau_1(0010c0100) &= 0010c01 \end{aligned}$$

Write a *BNF* translation grammar (or scheme) G_1 for translation τ_1 . If necessary, change the source grammar. Test grammar G_1 by drawing the (source and destination) trees of the sample $\tau_1(01c10) = 01c1$.

- (b) Consider the syntactic translation τ_2 of the source language $L(G)$ that outputs the palindromes and strips off the longest inner substring (if any) of bits 0 occurring immediately on the right of the centre c . One example suffices:

$$\tau_2(0100c0010) = 0100c10$$

Write a *BNF* translation grammar (or scheme) G_2 for translation τ_2 . If necessary, change the source grammar. Test grammar G_2 by drawing the (source and destination) trees of the sample $\tau_2(0100c0010) = 0100c10$.

- (c) (optional) Say if the translation grammar (or scheme) G_1 written before is deterministic or not, and justify your answer. Say if the translation τ_2 defined before is deterministic or not (independently of the grammar G_2 written for it), and justify your answer.

Solution

- (a) Here is a possible *BNF* grammar G_1 , with two nonterminals (axiom S):

$$G_1 \left\{ \begin{array}{l} 1: S \rightarrow \frac{0}{0} S \frac{0}{\varepsilon} \\ 2: S \rightarrow \frac{1}{1} X \frac{1}{1} \\ 3: S \rightarrow \frac{c}{c} \\ \hline 4: X \rightarrow \frac{1}{1} X \frac{1}{1} \\ 5: X \rightarrow \frac{0}{0} X \frac{0}{0} \\ 6: X \rightarrow \frac{c}{c} \end{array} \right. \quad \left| \quad \begin{array}{c} S \\ \swarrow \quad \downarrow \quad \searrow \\ \frac{0}{0} \quad S \quad \frac{0}{\varepsilon} \\ \swarrow \quad \downarrow \quad \searrow \\ \frac{1}{1} \quad X \quad \frac{1}{1} \\ \downarrow \\ \frac{c}{c} \end{array}$$

The (overlapped) source and destination sample syntax trees are drawn aside.

- (b) Here is a possible *BNF* grammar G_2 , with three nonterminals (axiom S):

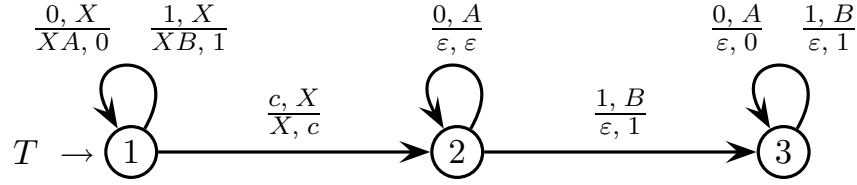
$$G_2 \left\{ \begin{array}{l} 1: S \rightarrow X_1 \\ 2: S \rightarrow X_2 \\ \hline 3: X_1 \rightarrow \frac{0}{0} X_1 \frac{0}{0} \\ 4: X_1 \rightarrow \frac{1}{1} X_1 \frac{1}{1} \\ 5: X_1 \rightarrow \frac{1}{1} X_2 \frac{1}{1} \\ \hline 6: X_2 \rightarrow \frac{0}{0} X_2 \frac{0}{\varepsilon} \\ 7: X_2 \rightarrow \frac{c}{c} \end{array} \right. \quad \left| \quad \begin{array}{c} S \\ \downarrow \\ X_1 \\ \swarrow \quad \downarrow \quad \searrow \\ \frac{0}{0} \quad X_1 \quad \frac{0}{0} \\ \swarrow \quad \downarrow \quad \searrow \\ \frac{1}{1} \quad X_2 \quad \frac{1}{1} \\ \swarrow \quad \downarrow \quad \searrow \\ \frac{0}{0} \quad X_2 \quad \frac{0}{\varepsilon} \\ \swarrow \quad \downarrow \quad \searrow \\ \frac{0}{0} \quad X_2 \quad \frac{0}{\varepsilon} \\ \downarrow \\ \frac{c}{c} \end{array}$$

The (overlapped) source and destination sample syntax trees are drawn aside.

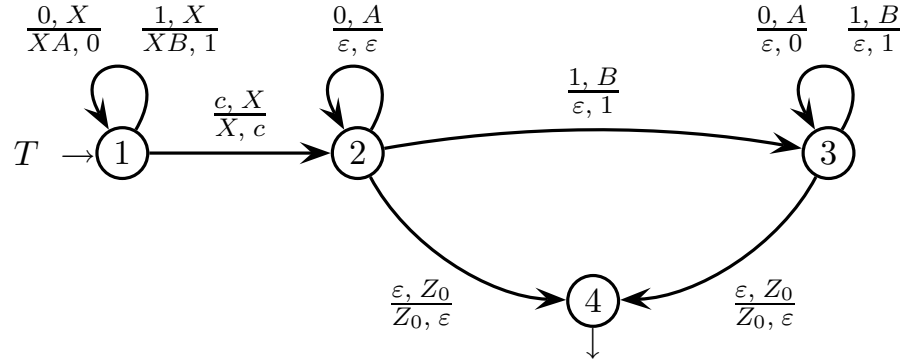
- (c) Grammar G_1 is deterministic, because its source grammar is of type $LL(1)$, as the groups of alternative rules 1, 2, 3 and 4, 5, 6 have different guide terminals.

Translation τ_2 is deterministic, as it has a deterministic pushdown translator T with few finite states. Here is a sketch thereof: in the initial state, T reads, pushes and emits all the input bits '0' and '1' (if any), until it reads the centre c (emitted yet not pushed), and then it enters another state; here T matches with the input all the stack symbols 0 (if any) and pops them, until it matches a stack symbol 1 (popped and emitted), and then it enters another state again; here T matches with the input all the stack symbols, pops them and emits them. If T stops in a non-initial state with input finished and empty stack, the translation is valid; if T stops in any other circumstance, it is invalid. This behaviour is fully deterministic. The reader may wish to design such a translator T by himself.

For completeness, here is a possible version of translator T . The stack alphabet is $\Gamma = \{ A, B, Z_0 \}$, where the symbols A and B encode the bits ‘0’ and ‘1’, respectively, the (unmovable) symbol Z_0 represents the stack bottom:



and the letter X represents any element of the stack alphabet (A , B or Z_0). Initially the stack is empty, and recognition may occur in the states 2 or 3 by empty stack. A version with recognition by final state (state 4) is the following:



Notice that grammar G_2 is not of type $LL(1)$, because of the alternative rules 4 and 5, which have the same guide source terminal ‘1’; also the alternative rules 1 and 2 infringe the $LL(1)$ property. It might be of type $LL(k)$ ($k \geq 2$), ELL or ELR , but answering rigorously wants a longer exam (i.e., recoding the grammar as a machine net possibly in the postfix form), here not requested. Otherwise, with some effort the reader may wish to write a deterministic grammar for translation τ_2 , by inspiring himself to translator T .

2. Consider the expressions defined by the following abstract syntax (axiom S):

$$\begin{cases} 1: S \rightarrow a & \text{--- } a \text{ is an atom (constant or variable)} \\ 2: S \rightarrow S \text{ op } S & \text{--- } \text{op} \text{ is a generic binary operator} \\ 3: S \rightarrow ' (' S ') ' & \text{--- parenthesized subexpression} \end{cases}$$

We want to verify the following two constraints that an expression should satisfy:

- in total, an expression may not contain more than five atoms
- in any position, an expression may not contain more than two pairs of immediately nested parentheses

Examples of semantically correct and incorrect expressions:

<i>expression</i>	<i>correct</i>	<i>reason</i>
$a \text{ op } (a \text{ op } a \text{ op } a)$	yes	
$a \text{ op } (a \text{ op } ((a \text{ op } a)))$	yes	
$(a \text{ op } a \text{ op } a) \text{ op } (a \text{ op } (a \text{ op } a))$	no	has $6 > 5$ atoms
$(a \text{ op } (a \text{ op } a)) \text{ op } (((a \text{ op } (a \text{ op } a))))$	no	has $6 > 5$ atoms and has 3 pairs of immediately nested parentheses

Attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>meaning</i>
<i>min</i>	left	integer	number of pairs of immediately nested parentheses in the (sub)tree rooted at S
<i>na</i>	left	integer	number of atoms in the (sub)tree rooted at S
<i>OK</i>	left	boolean	true if the (sub)expression rooted at S satisfies both the constraints listed above, false otherwise

Answer the following questions:

- Write an attribute grammar (in the table prepared on the next page), based on the above syntax and using only the three attributes listed above, that computes the value of the attribute *OK* in the tree root.
- Decorate with the attribute values the sample syntax tree (prepared on the next page) of the syntactically valid yet semantically *incorrect* expression below:

$$(a \text{ op } (a \text{ op } a)) \text{ op } (((a \text{ op } (a \text{ op } a))))$$

which is the last (fourth) sample expression listed above.

Attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>meaning</i>
<i>min</i>	left	integer	number of pairs of immediately nested parentheses in the (sub)tree rooted at S
<i>na</i>	left	integer	number of atoms in the (sub)tree rooted at S
<i>OK</i>	left	boolean	true if the (sub)expression rooted at S satisfies both the constraints listed above, false otherwise

attribute grammar to write - question (a)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1: $S_0 \rightarrow a$

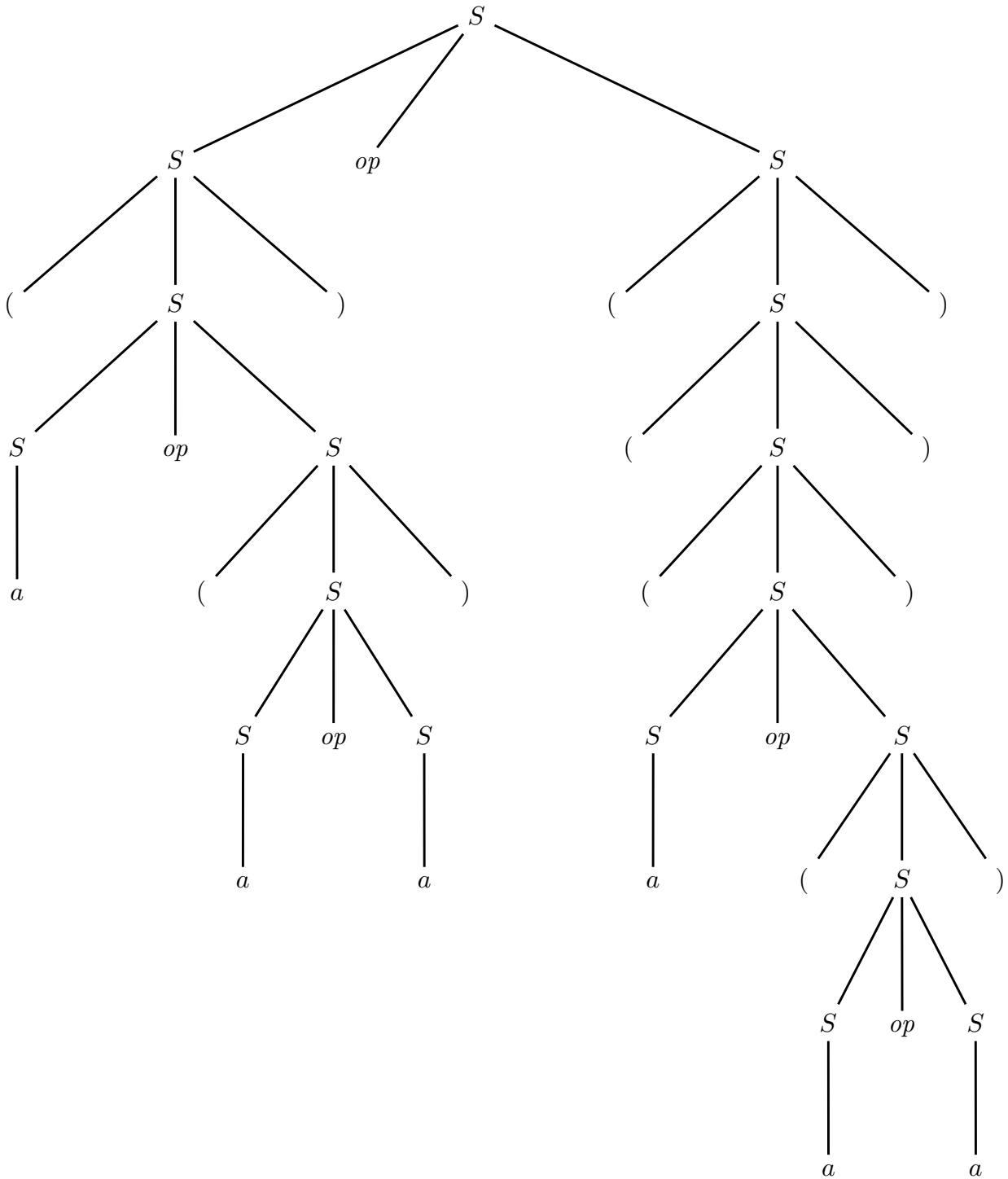
2: $S_0 \rightarrow S_1 \text{ op } S_2$

3: $S_0 \rightarrow '(S_1)'$

syntax tree of the (semantically incorrect) expression

$$(a \text{ op } (a \text{ op } a)) \text{ op } ((a \text{ op } (a \text{ op } a)))$$

to be decorated with all the attribute values - question (b)



Solution

(a) Here is a possible attribute grammar, which uses only the assigned attributes:

#	<i>syntax</i>	<i>semantics</i>	<i>comment</i>
1:	$S_0 \rightarrow a$	$min_0 = 0$ $na_0 = 1$ $OK_0 = true$	— initialized — initialized — initialized
2:	$S_0 \rightarrow S_1 \text{ op } S_2$	$min_0 = 0$ $na_0 = na_1 + na_2$ $OK_0 = OK_1 \wedge OK_2 \wedge (na_0 \leq 5)$	— reinitialized — computed — computed
3:	$S_0 \rightarrow '(' S_1 ')'$	$min_0 = min_1 + 1$ $na_0 = na_1$ $OK_0 = OK_1 \wedge (min_0 \leq 2)$	— computed — propagated — computed

All the attributes are (necessarily) associated to the (unique) nonterminal S . The attribute grammar is acyclic by construction and it is reasonably correct: the computation on the sample tree of question (b) is a sufficient test.

Furthermore, the attribute grammar is purely synthesized. Each one of the two attributes min and na depends only on itself, so they are independent, whereas the attribute OK depends on itself and also on both the attributes min and na . Thus clearly, when visiting a tree node of type 2 or 3, its attributes min and na have to be evaluated first, and its attribute OK has to be evaluated last (the divider line marks this difference). A tree node of type 1 consists only of constant initializations and the evaluation order of its attributes is arbitrary.

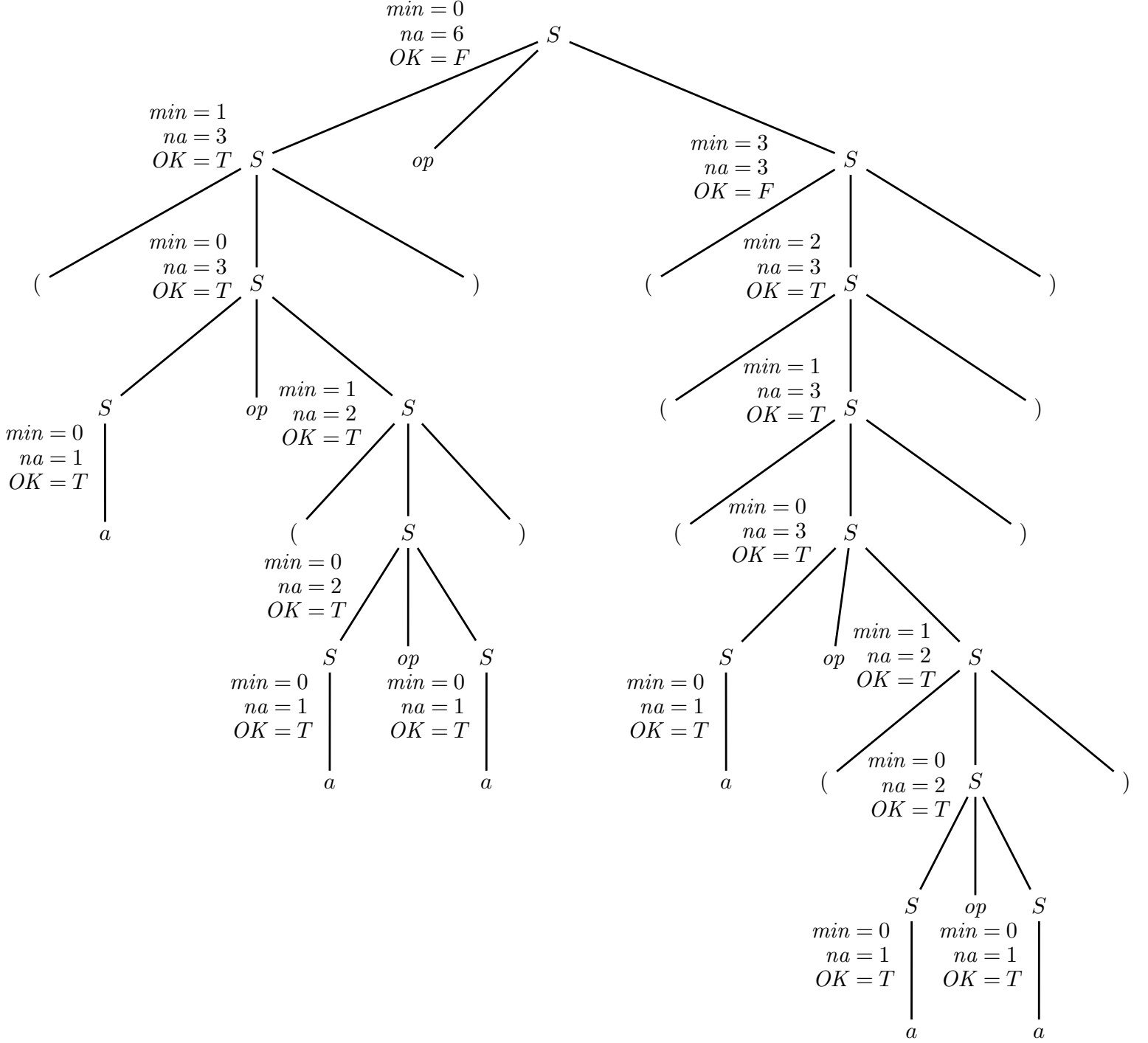
The interpretation of the semantic functions directly reflects the definition of the attributes. Notice that the attribute min , which counts the pairs of immediately nested parentheses, is reinitialized whenever a chain of such pairs is discontinued, that is, when a chain of nodes of type 3 is interrupted by a node of type 2.

Of course the attribute grammar is of type one-sweep, as it is purely synthesized. As such, the evaluation order of the child nodes in the rule 2, which has two brother subtrees, is arbitrary. Yet the syntactic support is ambiguous, as rule 2 has a two-side recursion, thus it is not deterministic. Therefore it is impossible to integrate semantic analysis and syntax analysis.

(b) Here is the syntax tree of the (semantically incorrect) expression:

$$(a \text{ op } (a \text{ op } a)) \text{ op } \left(\left((a \text{ op } (a \text{ op } a)) \right) \right)$$

decorated with all the attribute values (on the left of each inner node):



Attribute OK in the root is false, as expected.